

PART 1 INTERVIEW CHEAT SHEET

Senior / Staff ML Engineering

Before the Model Learns: The Journey of a Trustworthy ML Run

Memory rule: PLAN -> PLACE -> REPEAT -> RECORD -> STREAM -> PROVE

A credible result explains the request, machine, randomness, lineage, data movement, and output integrity.

1. PLAN

Functions 001-002

Config is the contract.

- Load YAML into a typed immutable config.
- Apply controlled environment overrides.
- Reject unknown fields and invalid values early.

Interview probe: Why fail before training?

2. PLACE

Functions 003

Ask the machine what it can do.

- Resolve auto to CUDA, MPS, or CPU.
- Explicit CUDA/MPS requests fail if unavailable.
- Never hide a device change.

Interview probe: Why is silent fallback risky?

3. REPEAT

Functions 004

Control randomness, not the universe.

- Seed Python, NumPy, PyTorch, and DataLoader generators.
- Request deterministic algorithms where supported.
- State platform and version limits.

Interview probe: Why is one seed not enough?

4. RECORD

Functions 005-008

Give each run a passport and a room.

- Capture environment, Git, command, model, and data versions.
- Create a unique run ID and isolated directory.
- Snapshot the resolved config actually used.

Interview probe: Can you reconstruct the run?

5. STREAM

Functions 009-010

Move records one at a time.

- Store one JSON object per line.
- Write atomically; read with yield.
- Report exact line errors and validate records.

Interview probe: Generator vs DataLoader?

6. PROVE

Functions 011-012

Fingerprint and inventory outputs.

- Hash files in binary chunks with SHA-256.
- Record path, type, size, checksum, and provenance.
- Declare expected outputs so missing files are visible.

Interview probe: Checksum vs manifest?

STAFF-LEVEL LENS: DO NOT STOP AT 'IT WORKS'

Decision

Why this abstraction, rather than something simpler or heavier?

Failure mode

What silently breaks: stale data, nondeterminism, partial writes, missing outputs?

Tradeoff

What did validation, determinism, or integrity checks cost?

Scale path

How does this evolve to shared storage, tracking servers, and distributed workers?

Ownership

What policy controls secrets, retention, schema drift, and untraceable runs?

One-breath chain: Config -> Device -> Seed -> Run identity -> Snapshot -> JSONL -> Checksum -> Manifest.

SENIOR / STAFF INTERVIEW PRESSURE TEST

12 high-probability probes

Use CRAFT: Context -> Risk -> Architecture -> Failure modes -> Tradeoff / scale path

1. How do you make an ML experiment reproducible?

Do not answer only 'set a seed.' Cover code commit, resolved config, data/model versions, environment, random generators, artifacts, and known nondeterminism limits.

2. Is deterministic execution always guaranteed?

No. Identical seeds do not guarantee identical results across PyTorch releases, platforms, CPU/GPU backends, or nondeterministic kernels. Determinism can also reduce performance.

3. How do you run locally on a Mac and later on CUDA?

Use an explicit device abstraction. Auto may select the best available backend; an explicit request should fail rather than silently changing intent. Keep device logic at boundaries.

4. What belongs in an experiment snapshot?

Resolved config, command, Git commit and dirty state, Python/packages, hardware/backend, data and model identifiers, metrics, checkpoints, predictions, and notes.

5. Why JSONL instead of one giant JSON array?

JSONL supports record-level streaming, exact line failures, simple inspection, and constant-memory iteration. It is not automatically a DataLoader or a random-access index.

6. Generator, IterableDataset, or DataLoader?

Generator/yield is the storage stream. IterableDataset adapts it to PyTorch. DataLoader adds batching, workers, collation, and prefetching. Multi-worker streams must be sharded.

7. Why atomic writes?

Write to a temporary file and replace the destination only after success. A crash should leave the previous valid file or no final file - never a partially valid dataset.

8. Checksum versus artifact manifest?

A checksum proves the bytes of one file. A manifest explains expected artifacts, paths, types, sizes, checksums, provenance, and missing outputs.

9. How do you prevent thousands of useless run folders?

Delete temporary development checks. Retain meaningful baselines, ablations, training, evaluation, and diagnostic failures. Add retention, promotion, and archival policies at scale.

10. Why not start immediately with MLflow, W&B, or DVC?

A small local project benefits from transparent primitives and no service overhead. Adopt a platform when teams need search, comparison UI, remote storage, concurrency, access control, and governance.

11. How would you scale this to distributed training?

Add rank-aware seeding, sharded samplers or streams, immutable data manifests, centralized metadata, object storage, transactional registration, and one authoritative run owner.

12. What would you never log?

Secrets, access tokens, uncontrolled full Git diffs, or private raw data. Record identifiers and lineage without turning the tracking layer into a data-leak channel.

Interview habit: state the failure mode and scaling path even when the question sounds like a simple coding question.

RECALL PAGE: SAY IT LIKE A STAFF ENGINEER

Part 1 closeout

Your project story, red flags, and research-backed interview themes

30-SECOND INTERVIEW ANSWER

Before any training run, I load and validate a typed configuration, resolve the available device without silent fallbacks, and seed Python, NumPy, PyTorch, and DataLoader randomness. I assign a unique run ID and snapshot the resolved config, Git state, environment, model, and dataset versions. Data moves through atomic JSONL files and can be streamed record by record. Every output is SHA-256 fingerprinted and indexed in a provenance manifest. That gives me reproducibility, auditability, and a clean path from local development to shared infrastructure.

CRAFT ANSWER PATTERN

C - Context	Scale, hardware, data, cost, and reliability constraints.
R - Risk	What could silently invalidate the result or block reproduction?
A - Architecture	Which components and contracts prevent those failures?
F - Failure modes	How do partial writes, stale data, duplicate workers, or missing artifacts surface?
T - Tradeoff / scale	What does rigor cost, and when do you add MLflow, object storage, CI/CD, or distributed workers?

12-FUNCTION RECALL MAP

001-002	Plan	Load and validate typed config
003	Place	Resolve CPU / MPS / CUDA
004	Repeat	Seed all random sources
005-008	Record	Environment, ID, directory, snapshot
009-010	Stream	Atomic JSONL write + generator read
011-012	Prove	SHA-256 + artifact manifest

RED-FLAG ANSWERS

- "I set random.seed(42), so every result is identical."
- "If CUDA is missing, I silently use CPU."
- "Saving the model file is enough to reproduce the run."
- "We load the entire dataset into RAM."
- "The filename tells us the data version."
- "We keep every run forever because storage is cheap."

WHAT THE INTERNET SCAN REPEATEDLY SURFACED

Current Staff/Senior ML interview guides repeatedly ask about reproducibility, data quality, experiment tracking, large-data pipelines, lineage, system failure modes, and build-vs-buy decisions.

Official Google Cloud MLOps and MLflow guidance treats configuration, code/data versions, metrics, metadata, and artifacts as first-class run components. PyTorch explicitly warns that complete reproducibility is not guaranteed across releases or platforms.

Research basis: PyTorch reproducibility docs; Google Cloud MLOps; MLflow Tracking; current Staff/Senior ML interview guides.

Final mantra: Reproducible runs. Traceable artifacts. Explicit tradeoffs.